
QML & C++ Entegrasyonu

Kapsamlı Uygulama Rehberi

Qt 6 • Modern CMake • Bol Çalışan Örnek

Bu belge, bir Qt uygulamasında **C++ arka ucu** ile **QML arayüzü** arasındaki köprüyü baştan sona anlatır: tip kaydı, özellikler (Q_PROPERTY), çağrılabilir metotlar, sinyal-slot bağları, enum'lar, tekil nesneler (singleton), liste modelleri ve iki yönlü iletişim. Her başlık **çok sayıda çalışan kod örneğiyle** desteklenmiştir.

Kapsam: Meta-Object sistemi • CMake & qt_add_qml_module • QML_ELEMENT ile tip kaydı • Q_PROPERTY & NOTIFY • Q_INVOKABLE & slotlar • sinyaller ve Connections • Q_ENUM • context property • singleton'lar • QAbstractListModel • C++'tan QML'e erişim • değer tipleri • yaygın hatalar

Hazırlanma Tarihi: 9 Temmuz 2026

Qt 6.x • C++17 • QtQuick

İçindekiler

1. Giriş: QML ile C++ Neden Birlikte Kullanılır?

Qt Quick uygulamalarında görev dağılımı nettir: **QML** akıcı, animasyonlu ve deklaratif kullanıcı arayüzünü tanımlar; **C++** ise iş mantığını, veri işlemeyi, dosya/ağ erişimini ve performans kritik hesaplamaları üstlenir. İkisini birbirine bağlayan tutkal, Qt'nin *Meta-Object* sistemidir.

Bilgi

Neden ayrım? QML yorumlanan (JavaScript motoru üzerinde çalışan) bir dildir; hızlı prototipleme ve arayüz için idealdir ama ağır hesaplama için uygun değildir. C++ ise derlenir ve çok hızlıdır. Doğru mimari: *ince* bir QML arayüz katmanı + *kalın* bir C++ mantık katmanı. Bu belge, bu iki katmanın nasıl konuşturulacağını gösterir.

1.1 Meta-Object Sistemi: Köprünün Temeli

C++ normalde derleme sonrası tip bilgisini kaybeder (*static typing*). QML'in çalışma zamanında bir C++ nesnesinin özelliklerini okuması, metotlarını çağırması ve sinyallerine bağlanması için **çalışma zamanı tip bilgisine (introspection)** ihtiyaç vardır. Qt bunu **QObject** taban sınıfı ve **moc** (Meta-Object Compiler) ile sağlar.

Bileşen	Görevi
QObject	Sinyal/slot, özellik ve nesne ağacının temel taban sınıfı. QML'e açılacak her tip bundan türer.
Q_OBJECT makrosu	Sınıfa meta-object desteği ekler. moc bu makroyu görünce ek kod üretir.
moc	Başlık dosyalarını tarar, Q_OBJECT içeren sınıflar için moc_*.cpp üretir. CMake bunu otomatik çalıştırır.
QML Engine	QML dosyalarını okur, JavaScript'i çalıştırır ve meta-object üzerinden C++ ile konuşur.

Dikkat

QML'e açtığınız her C++ sınıfı üç kurala uymalıdır: **(1) QObject**'ten (veya türevinden) türemeli, **(2)** sınıfın en başında Q_OBJECT makrosu bulunmalı, **(3)** varsayılan (argümentsiz) kurucusu olmalı veya tekil/oluşturulamaz olarak işaretlenmeli. Bu üçü olmadan QML tarafı sınıfı göremez.

1.2 İki Yönlü İletişimin Dört Yolu

C++ ile QML arasındaki bilgi akışı dört mekanizmayla olur. Bu belgenin tamamı bu dört yolu derinleştirir:

Mekanizma	Ne işe yarar	Yön
Q_PROPERTY	Veri açar; QML okur/yazar, değişince otomatik güncellenir	C++ ↔ QML
Q_INVOKABLE / slot	C++ metodunu QML'den çağırma	QML → C++
Sinyal (signal)	C++'ta olay olunca QML'i haberdar etme	C++ → QML
Metot/özellik erişimi	C++'tan QML nesnesine erişip metot çağırma	C++ → QML

2. Proje Kurulumu: CMake ve QML Modülü

Qt 6'da önerilen yol, C++ tiplerini bir **QML modülü** içinde toplamaktır. `qt_add_qml_module` komutu, tip kaydını otomatikleştirir ve QML derleyici (`qmlcache`, `qmltc`) optimizasyonlarını mümkün kılar.

2.1 Klasör Yapısı

```

1 MyApp/|—
2   CMakeLists.txt|—
3   main.cpp|—
4   backend.h      # C++ 111snf (şıbalk)|—
5   backend.cpp    # C++ 111snf (uygulama)|—
6   Main.qml       # Arayüz

```

2.2 CMakeLists.txt — Tam Örnek

```

1 cmake_minimum_required(VERSION 3.16)
2 project(MyApp VERSION 1.0 LANGUAGES CXX)
3
4 # Qt 6 Quick modülünü bul
5 find_package(Qt6 6.5 REQUIRED COMPONENTS Quick)
6
7 # AUTOMOC/AUTORCC ve C++ standardini ayarlar
8 qt_standard_project_setup(REQUIRES 6.5)
9
10 # Yurutulebilir hedef
11 qt_add_executable(appMyApp main.cpp)
12
13 # QML modülü: URI = "MyApp", surum = 1.0
14 qt_add_qml_module(appMyApp
15     URI MyApp
16     VERSION 1.0
17     QML_FILES
18         Main.qml
19     SOURCES
20         backend.h
21         backend.cpp
22 )
23
24 target_link_libraries(appMyApp PRIVATE Qt6::Quick)

```

URI nedir?

URI `MyApp` satırı, QML tarafında `import MyApp` yazarak bu modüldeki tüm C++ tiplerine erişebileceğiniz anlamına gelir. Modül adı ile `import` ifadesi **birebir aynı** olmalıdır.

2.3 main.cpp — Motoru Başlatma

```

1 #include <QGuiApplication>
2 #include <QQmlApplicationEngine>
3
4 int main(int argc, char *argv[])

```

```

5 {
6     QGuiApplication app(argc, argv);
7
8     QQmlApplicationEngine engine;
9
10    // Modul içindeki ana QML dosyasını yükle:
11    // "MyApp" modülü + "Main" tipi
12    engine.loadFromModule("MyApp", "Main");
13
14    if (engine.rootObjects().isEmpty())
15        return -1; // QML yüklenemedi
16
17    return app.exec();
18 }

```

İpucu

`loadFromModule("MyApp", "Main")` — Qt 6.5+ ile gelen modern yükleme biçimidir. Eski `engine.load(QUrl("qrc:/..."))` yönteminin yerini alır; dosya yollarıyla uğraşmanıza gerek kalmaz.

3. C++ Tiplerini QML'e Kaydetme

Bir C++ sınıfını QML'de kullanabilmek için önce onu *kaydetmek* gerekir. Qt 6'da bu, başlık dosyasına konan basit makrolarla yapılır; `moc` ve `qt_add_qml_module` gerisini halleder.

3.1 QML_ELEMENT — En Yaygın Yöntem

Sınıfa `QML_ELEMENT` makrosunu ekleyin; sınıf, C++'taki adıyla QML'de kullanılabilir hâle gelir.

`</>` backend.h — Kaydedilebilir bir sınıf

```

1  #ifndef BACKEND_H
2  #define BACKEND_H
3
4  #include <QObject>
5  #include <QQmlEngine> // QML_ELEMENT için gerekli
6
7  class Backend : public QObject
8  {
9      Q_OBJECT
10     QML_ELEMENT // <-- Bu sinifi QML'e "Backend" olarak kaydeder
11
12 public:
13     // QML'in nesne oluşturabilmesi için argumansız kurucu sart
14     explicit Backend(QObject *parent = nullptr);
15
16     Q_INVOKABLE QString greet(const QString &name) const;
17 };
18
19 #endif

```

```
1 // backend.cpp
```

```

2 #include "backend.h"
3
4 Backend::Backend(QObject *parent) : QObject(parent) {}
5
6 QString Backend::greet(const QString &name) const
7 {
8     return QStringLiteral("Merhaba, %1!").arg(name);
9 }

```

Artık QML tarafında hiçbir ek işlem olmadan kullanılabilir:

```

1 import QtQuick
2 import MyApp           // CMake'teki URI ile aynı
3
4 Window {
5     width: 300; height: 200; visible: true
6
7     Backend { id: backend } // C++ sınıfından nesne olusturuldu!
8
9     Text {
10         anchors.centerIn: parent
11         text: backend.greet("Dunya") // -> "Merhaba, Dunya!"
12     }
13 }

```

Bilgi

QML_ELEMENT, sınıfı C++ ismiyle kaydeder. Farklı bir isim istiyorsanız QML_NAMED_ELEMENT(BaskaIsim) kullanın. Kaydın çalışması için sınıfın bir QML modülünün parçası olması (yani SOURCES altında listelenmesi) yeterlidir; ayrı bir qmlRegisterType çağrısına gerek yoktur.

3.2 Kayıt Makroları Karşılaştırması

Makro	Ne zaman kullanılır
QML_ELEMENT	Sınıfı C++ adıyla açar. Standart, en sık kullanılan.
QML_NAMED_ELEMENT(Ad)	QML'de farklı bir isim vermek için.
QML_SINGLETON	Tek örnekli (singleton) tip. Bölüm 8'e bakın.
QML_UNCREATABLE("msg")	QML'de newlenemez ama tipi/enum'u görünür.
QML_ANONYMOUS	İsimsiz; sadece özellik tipi olarak döner, QML'de yaratılamaz.
QML_FOREIGN	Değiştiremediğiniz (3. parti/Qt) bir tipi kaydetmek için.

3.3 Eski Yöntem: qmlRegisterType (Qt 5 tarzı)

Eski projelerde veya modül kullanmadığınızda kaydı main.cpp içinden elle yaparsınız. Bilmekte fayda var:

```

1 #include "backend.h"
2 // ...
3 int main(int argc, char *argv[])
4 {

```

```

5   QApplication app(argc, argv);
6
7   // qmlRegisterType<Sinif>("Uri", surumBuyuk, surumKucuk, "QmlAdi");
8   qmlRegisterType<Backend>("MyApp", 1, 0, "Backend");
9
10  QQmlApplicationEngine engine;
11  engine.load(QUrl(QStringLiteral("qrc:/Main.qml")));
12  return app.exec();
13 }

```

⚠ Dikkat

Yeni projelerde `qmlRegisterType` yerine `QML_ELEMENT` + modül yaklaşımını tercih edin. Elle kayıt; sürüm numaralarını manuel yönetmenizi gerektirir, derleme zamanı kontrolü sağlamaz ve QML derleyici optimizasyonlarından yararlanamaz.

4. Q_PROPERTY: Veri Açmanın Kalbi

`Q_PROPERTY`, bir C++ üye değişkenini QML'e *özellik* olarak açar. QML bunu okuyabilir, yazabilir ve — en önemlisi — değeri değiştiğinde arayüz **otomatik olarak** güncellenir. Bu, QML'in *property binding* (özellik bağlama) gücünün C++ tarafındaki karşılığıdır.

4.1 Anatomi: READ, WRITE, NOTIFY

```
1 Q_PROPERTY(type name READ getter WRITE setter NOTIFY nameChanged)
```

Anahtar

Açıklama

READ	Değeri döndüren getter metodu. Zorunlu (MEMBER yoksa).
WRITE	Değeri ayarlayan setter. Opsiyonel — yoksa salt-okunur.
NOTIFY	Değer değişince yayılan sinyal. Binding güncellemesi için şart .
CONSTANT	Değer hiç değişmez; NOTIFY gerekmez.
MEMBER	Getter/setter yazmadan üyeyi doğrudan açar.
REQUIRED	QML nesnesi oluşturulurken bu özellik atanmak zorunda.

4.2 Tam Örnek: Sayaç Sınıfı

```
</> counter.h
```

```

1 #ifndef COUNTER_H
2 #define COUNTER_H
3
4 #include <QObject>
5 #include <QQmlEngine>
6
7 class Counter : public QObject
8 {

```

```

9      Q_OBJECT
10     QML_ELEMENT
11
12     // "count" adında, okunabilir/yazılabilir bir int özellik
13     Q_PROPERTY(int count READ count WRITE setCount NOTIFY countChanged)
14
15 public:
16     explicit Counter(QObject *parent = nullptr) : QObject(parent) {}
17
18     int count() const { return m_count; }
19
20     void setCount(int value)
21     {
22         if (m_count == value) // Değer değişmediyse çık (sonsuz döngüyü önler)
23             return;
24         m_count = value;
25         emit countChanged(); // QML'e "degistim" de
26     }
27
28     Q_INVOKABLE void increment() { setCount(m_count + 1); }
29     Q_INVOKABLE void reset()     { setCount(0); }
30
31 signals:
32     void countChanged(); // NOTIFY sinyali
33
34 private:
35     int m_count = 0;
36 };
37
38 #endif

```

⚠ En kritik iki kural

1. Setter içinde *daima* değer değişti mi diye kontrol edin. Bu kontrol olmadan binding'ler kendini tetikleyip **sonsuz döngüye** girebilir.
2. Değeri değiştiren her yolda NOTIFY sinyalini emit edin. Etmezseniz QML arayüzü *sessizce eski değeri gösterir* ve saatlerce hata ararsınız.

4.3 QML Tarafı: Binding Sihiri

```

1  import QtQuick
2  import QtQuick.Controls
3  import MyApp
4
5  Window {
6      width: 300; height: 200; visible: true
7
8      Counter { id: counter }
9
10     Column {
11         anchors.centerIn: parent
12         spacing: 12
13
14         // counter.count degistiginde bu Text OTOMATIK guncellenir
15         Text {
16             text: "Sayac: " + counter.count

```

```

17         font.pixelSize: 24
18     }
19
20     Button {
21         text: "Arttir (+1)"
22         onClicked: counter.increment() // C++ metodu cagrildi
23     }
24     Button {
25         text: "Sifirla"
26         onClicked: counter.count = 0 // WRITE ile dogrudan atama
27     }
28 }
29

```

💡 İpucu

Burada `text: "Sayac: " + counter.count` bir **binding**'dir. QML motoru `count`'un `countChanged` sinyali otomatik abone olur; sinyal her yayıldığında `Text` kendini yeniler. Siz elle güncelleme kodu yazmazsınız — bildirimsel (declarative) programlamanın özü budur.

4.4 Kısayol: MEMBER ile Hızlı Özellik

Getter/setter yazmak istemediğiniz basit durumlarda `MEMBER` kullanın:

```

1 class Settings : public QObject
2 {
3     Q_OBJECT
4     QML_ELEMENT
5     // Getter/setter yok; uyeye dogrudan baglanir. NOTIFY yine onerilir.
6     Q_PROPERTY(QString username MEMBER m_username NOTIFY usernameChanged)
7 public:
8     explicit Settings(QObject *parent = nullptr) : QObject(parent) {}
9 signals:
10    void usernameChanged();
11 private:
12    QString m_username;
13 };

```

4.5 Salt-Okunur ve Sabit Özellikler

```

1 class AppInfo : public QObject
2 {
3     Q_OBJECT
4     QML_ELEMENT
5     // WRITE yok -> QML sadece okuyabilir
6     Q_PROPERTY(QString status READ status NOTIFY statusChanged)
7     // CONSTANT -> hic degismez, NOTIFY gerekmez
8     Q_PROPERTY(QString version READ version CONSTANT)
9 public:
10    explicit AppInfo(QObject *parent = nullptr) : QObject(parent) {}
11    QString status() const { return m_status; }
12    QString version() const { return QStringLiteral("1.0.0"); }
13 signals:
14    void statusChanged();
15 private:

```

```

16     QString m_status = QStringLiteral("Hazir");
17 };

```

5. Q_INVOKABLE ve Slotlar: C++ Metotlarını Çağırma

QML'in bir C++ metodunu çağırabilmesi için metodun meta-object sistemine kayıtlı olması gerekir. Bunun iki yolu vardır: Q_INVOKABLE ile işaretlemek veya metodu bir slot yapmak.

5.1 İki Yöntem, Aynı Sonuç

```

1  class Calculator : public QObject
2  {
3      Q_OBJECT
4      QML_ELEMENT
5  public:
6      explicit Calculator(QObject *parent = nullptr) : QObject(parent) {}
7
8      // Yontem 1: Q_INVOKABLE ile isaretle
9      Q_INVOKABLE double add(double a, double b) { return a + b; }
10     Q_INVOKABLE double divide(double a, double b)
11     {
12         if (b == 0.0)
13             return 0.0;      // Sifira bolme korumasi
14         return a / b;
15     }
16
17  public slots:
18      // Yontem 2: public slot -> QML'den otomatik cagrilabilir
19      QString format(double number)
20      {
21          return QString::number(number, 'f', 2);    // "3.14" gibi
22      }
23 };

```

```

1  Calculator { id: calc }
2
3  Text {
4      // Her ucu de QML'den dogrudan cagrilabilir
5      text: calc.format(calc.divide(calc.add(10, 5), 3)) // "5.00"
6  }

```

❏ Q_INVOKABLE mı, slot mu?

İkisi de QML'den çağrılabilir. **Slot**; ayrıca bir sinyale connect edilebildiği için sinyal-slot mimarisine uygundur. **Q_INVOKABLE** ise "bu sadece QML'den çağrılan bir metot" niyetini daha net belirtir. Modern tercih: sinyale bağlanmayacaksa Q_INVOKABLE kullanın.

5.2 Parametre ve Dönüş Tipleri

QML ile C++ arasında otomatik dönüştürülen tipler (QVariant üzerinden):

C++ Tipi	QML Karşılığı	Not
int, double, bool	int, real, bool	Doğrudan
QString	string	UTF-16
QVariant	var	Her şeyi taşır
QVariantList	list/array	JS dizisi
QVariantMap	object	JS nesnesi (anahtar-değer)
QObject*	(nesne)	Referans olarak geçer
QDateTime	date	Tarih/saat

5.3 JavaScript Nesnesi Döndürmek

QML'e yapılandırılmış veri döndürmenin pratik yolu `QVariantMap`'tir:

```

1 Q_INVOKABLE QVariantMap userInfo(int id)
2 {
3     QVariantMap m;
4     m["id"] = id;
5     m["name"] = QStringLiteral("Ayse");
6     m["age"] = 30;
7     m["active"] = true;
8     return m; // QML'de bir JS nesnesi olur
9 }

```

```

1 var u = backend.userInfo(42)
2 console.log(u.name, u.age) // "Ayse 30"

```

6. Sinyaller: C++'tan QML'i Haberdar Etmek

Özellikler "değer" iletmek için idealdir; ama "bir olay oldu" (indirme bitti, hata oluştu, bağlantı koptu) demek için **sinyaller** kullanılır. C++'ta sinyal yayarsınız, QML tarafında ona *handler* bağlarsınız.

6.1 C++ Tarafı: Sinyal Tanımlama ve Yayma

```

1 class Downloader : public QObject
2 {
3     Q_OBJECT
4     QML_ELEMENT
5 public:
6     explicit Downloader(QObject *parent = nullptr) : QObject(parent) {}
7
8     Q_INVOKABLE void start(const QString &url)
9     {
10         emit started(url);
11         // ... indirme mantigi (basitleştirilmiş) ...
12         for (int p = 0; p <= 100; p += 25)
13             emit progress(p); // Parametrelili sinyal
14         emit finished(url, true); // Çok parametrelili sinyal
15     }
16 }

```

```

17 signals:
18     void started(const QString &url);
19     void progress(int percent);
20     void finished(const QString &url, bool success);
21 };

```

6.2 QML Tarafı: on<Sinyal> Handler'ı

Qt otomatik olarak her sinyalAdı için onSinyalAdı handler'ı üretir (ilk harf büyük):

```

1 Downloader {
2     id: downloader
3
4     // "started" -> "onStarted"
5     onStarted: (url) => console.log("Basladi:", url)
6
7     // "progress" -> "onProgress"; parametre adi sinyaldeki ile ayni
8     onProgress: (percent) => progressBar.value = percent / 100
9
10    onFinished: (url, success) => {
11        if (success)
12            statusText.text = "Bitti: " + url
13        else
14            statusText.text = "Hata!"
15    }
16 }

```

6.3 Connections: Dışarıdan Sinyale Bağlanma

Nesnenin tanımlı olduğu yerde handler yazamıyorsanız (ör. context property ile gelen bir nesne) **Connections** kullanın:

```

1 Connections {
2     target: downloader // Hangi nesnenin sinyalleri
3     function onFinished(url, success) {
4         console.log("Connections ile yakalandi:", url)
5     }
6 }

```

💡 İpucu

Sinyal parametrelerine handler içinde **isimleriyle** erişebilirsiniz. Eski Qt 5 sözdiziminde bunlar sihirli biçimde kapsamdaydı; Qt 6'da modern ve önerilen biçim, yukarıdaki gibi **ok fonksiyonu** veya **function onX(param)** parametre listesidir. Bu, isim çakışmalarını önler.

7. Enum'ları QML'e Açmak

C++ enum'larını QML'de kullanmak, "sihirli sayılar" yerine anlamlı isimler kullanmayı sağlar. **Q_ENUM** makrosu enum'u meta-object sistemine kaydeder.

7.1 Q_ENUM ile Kayıt

```

1 class Player : public QObject
2 {
3     Q_OBJECT
4     QML_ELEMENT
5 public:
6     explicit Player(QObject *parent = nullptr) : QObject(parent) {}
7
8     enum State {
9         Stopped,
10        Playing,
11        Paused
12    };
13    Q_ENUM(State)      // <-- Enum'u QML'e acar
14
15    Q_PROPERTY(State state READ state WRITE setState NOTIFY stateChanged)
16
17    State state() const { return m_state; }
18    void setState(State s) {
19        if (m_state == s) return;
20        m_state = s;
21        emit stateChanged();
22    }
23 signals:
24     void stateChanged();
25 private:
26     State m_state = Stopped;
27 };

```

7.2 QML'de Enum Kullanımı

Enum değerlerine TipAdi.DegerAdi biçiminde erişilir:

```

1 Player {
2     id: player
3
4     onStateChanged: {
5         switch (state) {
6             case Player.Playing: console.log("Oynatiliyor"); break
7             case Player.Paused:  console.log("Duraklatildi"); break
8             case Player.Stopped: console.log("Durduruldu"); break
9         }
10    }
11 }
12
13 Button {
14     text: "Oynat"
15     // Sayi yerine anlamlı isim:
16     onClicked: player.state = Player.Playing
17 }

```

⚠ Dikkat

Enum değerlerine QML'de **sınıf adı** üzerinden erişilir (`Player.Playing`), nesne adı üzerinden değil (`player.Playing` **yanlıştır**). Ayrıca enum kaydı için sınıfın QML'e kayıtlı olması (`QML_ELEMENT`) gerekir; sadece enum'u açmak istiyor ama nesne oluşturulmasını istemiyorsanız `QML_UNCREATABLE` ile birleştirin.

8. Context Property: Global Nesne Enjeksiyonu

`QML_ELEMENT` ile tipleri açtınız — ama bazen QML'in *belirli bir C++ örneğine* (tip değil, hazır nesne) erişmesini istersiniz. Örneğin veritabanına bağlı tek bir backend nesnesi. Bunun klasik yolu **context property**'dir.

8.1 setContextProperty ile Nesne Enjekte Etme

```
1 #include <QmlContext>
2 #include "backend.h"
3
4 int main(int argc, char *argv[])
5 {
6     QGuiApplication app(argc, argv);
7     QmlApplicationEngine engine;
8
9     Backend backend; // C++'ta oluşturulan tek örnek
10    backend.setUser("admin");
11
12    // "backend" adıyla TUM QML dosyalarına global olarak enjekte et
13    engine.rootContext()->setContextProperty("backend", &backend);
14
15    engine.loadFromModule("MyApp", "Main");
16    return app.exec();
17 }
```

```
1 // Herhangi bir QML dosyasında, import bile gerekmeden:
2 Text {
3     text: "Kullanici: " + backend.user // doğrudan "backend"
4 }
```

⚠ Context property'nin dezavantajları

Context property'ler pratik görünse de modern Qt'de **önerilmez**: (1) Derleme zamanı tip kontrolü yoktur — yanlış özellik adı sadece çalışma zamanında sessizce `undefined` döner. (2) QML derleyici (`qmltc`) bunları optimize edemez. (3) Her QML dosyasında görünür olması kapsam kirliliği yaratır. Mümkünse yerine **singleton** (Bölüm 8) kullanın.

8.2 Alternatif: initialProperties ile Örnek Aktarımı

Tek bir kök nesneye örnek geçirmek için daha temiz bir yol:

```
1 Backend backend;
2 QmlApplicationEngine engine;
3 engine.loadFromModule("MyApp", "Main");
4 // Kök nesnenin "backend" özelliğine C++ örneğini bağla
```

```
5 // (Main.qml icinde: property Backend backend)
```

9. Singleton'lar: Uygulama Geneli Tek Nesne

Uygulama boyunca tek bir örneği olması gereken servisler (ayarlar yöneticisi, oturum, tema, veritabanı erişimi) için **singleton** idealdir. Context property'nin tip-güvenli, optimize edilebilir modern alternatiftir.

9.1 QML_SINGLETON ile Tanımlama

```
1 class AppSettings : public QObject
2 {
3     Q_OBJECT
4     QML_ELEMENT
5     QML_SINGLETON // <-- Tek ornek olacak
6     Q_PROPERTY(QString theme READ theme WRITE setTheme NOTIFY themeChanged)
7     Q_PROPERTY(int fontSize READ fontSize WRITE setFontSize NOTIFY fontSizeChanged)
8 public:
9     explicit AppSettings(QObject *parent = nullptr) : QObject(parent) {}
10
11     QString theme() const { return m_theme; }
12     void setTheme(const QString &t) {
13         if (m_theme == t) return;
14         m_theme = t; emit themeChanged();
15     }
16     int fontSize() const { return m_fontSize; }
17     void setFontSize(int s) {
18         if (m_fontSize == s) return;
19         m_fontSize = s; emit fontSizeChanged();
20     }
21 signals:
22     void themeChanged();
23     void fontSizeChanged();
24 private:
25     QString m_theme = QStringLiteral("dark");
26     int m_fontSize = 14;
27 };
```

9.2 QML'de Singleton Kullanımı

Singleton'a **tip** adıyla doğrudan erişilir; nesne oluşturmaya gerek yoktur:

```
1 import QtQuick
2 import MyApp
3
4 Window {
5     // Tum uygulamada ayni ornek. Nesne olusturmadan tip adiyla eris:
6     color: AppSettings.theme === "dark" ? "#222" : "#fff"
7
8     Text {
9         text: "Ayarlar"
10        font.pixelSize: AppSettings.fontSize // Global, canli baglanti
11        color: AppSettings.theme === "dark" ? "white" : "black"
12    }
13
14    Button {
```

```

15     text: "Temayı Değiştir"
16     onClicked: AppSettings.theme =
17         AppSettings.theme === "dark" ? "light" : "dark"
18 }
19 }

```

İpucu

Bir singleton'ın theme özelliğini bir yerde değiştirdiğinizde, uygulamadaki *tüm* binding'ler anında güncellenir. Bu, tema/dil/ayar gibi global durumları yönetmenin en zarif yoludur. C++ tarafında singleton örneğine engine üzerinden de erişebilirsiniz.

9.3 Özel Kurucu Gereken Singleton

Örneğin oluşumunu kontrol etmek isterseniz create fonksiyonu kullanın:

```

1 class Database : public QObject
2 {
3     Q_OBJECT
4     QML_ELEMENT
5     QML_SINGLETON
6 public:
7     // QML motoru bu statik fonksiyonu çağırarak örneği oluşturur
8     static Database *create(QQmlEngine *, QJSEngine *engine)
9     {
10         Database *db = new Database();
11         db->connectToDb();
12         return db; // Sahiplik QML motoruna geçer
13     }
14 private:
15     explicit Database(QObject *parent = nullptr) : QObject(parent) {}
16     void connectToDb() { /* ... */ }
17 };

```

10. Liste Modelleri: QAbstractListModel

Tek nesne değil de bir *liste* (kişiler, mesajlar, ürünler) göstermek istediğinizde, C++ verisini **ListView**'e bağlamanın performanslı yolu **QAbstractListModel** türetmektir.

10.1 Rol Tabanlı Model Mantığı

QML modelleri **rol (role)** kavramıyla çalışır. Her satır birden çok alana sahiptir (ad, e-posta, yaş); her alan bir "rol"dür. QML delegate'i bu rollere isimle erişir.

</> contactmodel.h

```

1 #ifndef CONTACTMODEL_H
2 #define CONTACTMODEL_H
3
4 #include <QAbstractListModel>
5 #include <QQmlEngine>
6
7 struct Contact {

```

```

8     QString name;
9     QString email;
10 };
11
12 class ContactModel : public QAbstractListModel
13 {
14     Q_OBJECT
15     QML_ELEMENT
16 public:
17     // Rollerini tanımla (Qt::UserRole'dan başlar)
18     enum Roles {
19         NameRole = Qt::UserRole + 1,
20         EmailRole
21     };
22
23     explicit ContactModel(QObject *parent = nullptr) : QAbstractListModel(parent)
24     {
25         m_data = {
26             {"Ali", "ali@ornek.com"},
27             {"Veli", "veli@ornek.com"}
28         };
29     }
30
31     // 1) Kac satir var?
32     int rowCount(const QModelIndex &parent = QModelIndex()) const override
33     {
34         Q_UNUSED(parent)
35         return m_data.count();
36     }
37
38     // 2) Belirli satir+rol icin veri
39     QVariant data(const QModelIndex &index, int role) const override
40     {
41         if (!index.isValid() || index.row() >= m_data.count())
42             return {};
43         const Contact &c = m_data.at(index.row());
44         switch (role) {
45             case NameRole: return c.name;
46             case EmailRole: return c.email;
47         }
48         return {};
49     }
50
51     // 3) Rol numaralarini QML'deki isimlere esle
52     QHash<int, QByteArray> roleNames() const override
53     {
54         return {
55             { NameRole, "name" },
56             { EmailRole, "email" }
57         };
58     }
59
60     // Satir eklemek: beginInsertRows/endInsertRows sart!
61     Q_INVOKABLE void addContact(const QString &name, const QString &email)
62     {
63         beginInsertRows(QModelIndex(), m_data.count(), m_data.count());
64         m_data.append({ name, email });
65         endInsertRows();
66     }

```

```

67
68 private:
69     QList<Contact> m_data;
70 };
71
72 #endif

```

⚠ Üç zorunlu metot + değişiklik bildirimi

rowCount, data ve roleNames **override edilmek zorundadır**. Ayrıca veriyi değiştirirken beginInsertRows/endInsertRows (ekleme), beginRemoveRows/endRemoveRows (silme) veya dataChanged (güncelleme) çağrılarını **atlamayın**. Atlarsanız **ListView** güncellenmez veya çökme yaşarsınız.

10.2 QML'de ListView ile Gösterme

```

1 import QtQuick
2 import QtQuick.Controls
3 import MyApp
4
5 ListView {
6     anchors.fill: parent
7     model: ContactModel {} // C++ modeli dogrudan bagla
8
9     delegate: Rectangle {
10         width: ListView.view.width
11         height: 50
12         color: index % 2 ? "#f0f0f0" : "white"
13
14         Row {
15             spacing: 10
16             anchors.verticalCenter: parent.verticalCenter
17             // roleNames'teki isimler dogrudan kapsamda:
18             Text { text: name; font.bold: true }
19             Text { text: email; color: "gray" }
20         }
21     }
22 }

```

📌 Basit listeler için kısayol

Sadece birkaç öğelik statik/basit bir liste için **QAbstractListModel** fazla ağır kalır. **QStringList** veya **QVariantList** döndüren bir **Q_PROPERTY** de model olarak kullanılabilir. Ancak büyük, sık değişen veya çok alanlı listeler için mutlaka **QAbstractListModel** tercih edin — performans farkı büyüktür.

11. C++'tan QML'e Erişmek

Şimdiye kadar akış çoğunlukla QML → C++ yönündeydi. Bazen tersine, C++'tan bir QML nesnesinin özelliğini okumak, metodunu çağırmak veya sinyaline bağlanmak istersiniz. Bu genelde *gerekli olmadıkça kaçınılması* gereken bir yöndür ama gerektiğinde şöyle yapılır.

11.1 Kök Nesneye ve Alt Nesnelere Erişim

```

1 QQmlApplicationEngine engine;
2 engine.loadFromModule("MyApp", "Main");
3
4 // Kök QML nesnesi (Window)
5 QObject *root = engine.rootObjects().first();
6
7 // Özellik oku/yaz (property adıyla)
8 root->setProperty("title", "C++ tarafından degistirildi");
9 QVariant w = root->property("width");
10
11 // objectName ile bir alt nesne bul
12 // (QML'de: Text { objectName: "statusText" })
13 QObject *label = root->findChild<QObject*>("statusText");
14 if (label)
15     label->setProperty("text", "Guncellendi");

```

11.2 C++'tan QML Fonksiyonu Çağırarak

QML'de function olarak tanımlı bir metodu `QMetaObject::invokeMethod` ile çağırabilirsiniz:

```

1 // Main.qml
2 Window {
3     function showMessage(text) {
4         console.log("QML fonksiyonu:", text)
5         return "alindi: " + text
6     }
7 }

```

```

1 QObject *root = engine.rootObjects().first();
2 QVariant result;
3 QMetaObject::invokeMethod(root, "showMessage",
4     Q_RETURN_ARG(QVariant, result),
5     Q_ARG(QVariant, QString("Selam QML")));
6 qDebug() << result; // "alindi: Selam QML"

```

Dikkat

C++'tan QML nesnelere `findChild/invokeMethod` ile erişmek **kırılgan** bir desendir: QML yapısını değiştirdiğinizde C++ kodu sessizce bozulur ve derleyici sizi uyarır. **Tercih edilen mimari:** veriyi ve mantığı C++'ta tutun, `Q_PROPERTY` ve sinyallerle QML'e *itin*; QML bunlara binding ile tepki versin. C++'ın QML'e uzanması yerine QML'in C++'a bağlanması daha sağlıklıdır.

12. Değer Tipleri ve Gadget'lar (Q_GADGET)

Her veri yapısının tam bir `QObject` olması gerekmez. Konum, renk, boyut gibi küçük, kopyalanabilir veri taşıyıcıları için `Q_GADGET` kullanılır — sinyal/slot yükü olmadan meta bilgi sağlar.

```

1 class Point3D
2 {
3     Q_GADGET // QObject değil! Hafif meta-tip
4     Q_PROPERTY(double x READ x WRITE setX)

```

```

5     Q_PROPERTY(double y READ y WRITE setY)
6     Q_PROPERTY(double z READ z WRITE setZ)
7 public:
8     double x() const { return m_x; } void setX(double v) { m_x = v; }
9     double y() const { return m_y; } void setY(double v) { m_y = v; }
10    double z() const { return m_z; } void setZ(double v) { m_z = v; }
11
12    Q_INVOKABLE double length() const
13    {
14        return std::sqrt(m_x*m_x + m_y*m_y + m_z*m_z);
15    }
16 private:
17     double m_x = 0, m_y = 0, m_z = 0;
18 };

```

Özellik	Q_OBJECT	Q_GADGET
Sinyal/slot	Var	Yok
Kopyalanabilir	Hayır (kimliği var)	Evet (değer tipi)
Bellek yükü	Yüksek	Düşük
Kullanım	Servis, model, mantık	Küçük veri taşıyıcı

13. Yaygın Hatalar ve En İyi Pratikler

13.1 Sık Yapılan Hatalar

Hata	Sonuç & Çözüm
NOTIFY sinyalini emit etmemek	QML eski değeri gösterir. Setter'da değeri değiştirince mutlaka emit edin.
Setter'da değişiklik kontrolü yapmamak	Binding sonsuz döngüye girer. <code>if (m == v) return;</code> ekleyin.
Q_OBJECT makrosunu unutmak	Sinyaller/özellikler çalışmaz, linker hatası. Sınıf başına ekleyin.
Argümentsiz kurucu olmaması	QML nesneyi oluşturamaz. Ya kurucu ekleyin ya <code>QML_UNCREATABLE</code> .
Modeli <code>begin/end...</code> olmadan değiştirmek	ListView bozulur/çöker. Değişiklik metodlarını sarmalayın.
Nesne sahipliğini yanlış yönetmek	Çift silme veya sızıntı. QML'e verdiğiniz nesnelerde JS sahipliğine dikkat.

13.2 Nesne Sahipliği (Ownership)

Bilgi

Bir C++ metodu QML'e `QObject*` döndürdüğünde sahiplik kuralı: nesnenin **parent'ı varsa** C++ sahiptir; **parent'ı yoksa** QML çöp toplayıcısı (garbage collector) onu silebilir. QML'in beklenmedik anda silmesini istemiyorsanız nesneye bir parent verin veya `QqmlEngine::setObjectOwnership(obj, QqmlEngine::CppOwnership)` ile sahipliği açıkça C++'a bırakın.

13.3 Mimari Tavsiyeleri

- **Tek yön akış:** Veri ve mantık C++'ta; QML sadece *sunum*. C++ QML'e veri iter, QML C++ metodlarını çağırır. C++'ın QML iç yapısına uzanmasından kaçının.
- **Singleton > context property:** Global servisler için tip-güvenli, optimize edilebilir singleton'ları tercih edin.
- **Modül kullanın:** `qt_add_qml_module + QML_ELEMENT`, elle `qmlRegisterType`'a yeğdir.
- **Ağır işi bloklamayın:** Uzun süren C++ işlemlerini ayrı bir iş parçacığında (`QThread`, `QtConcurrent`) yapın; bitince sinyalle QML'i haberdar edin. Aksi hâlde arayüz donar.
- **const & referans:** Metot parametrelerinde büyük tipleri `const QString&` gibi referansla alın; gereksiz kopyalamayı önler.

13.4 Hızlı Referans Tablosu

Makro / Üge	Amaç
<code>Q_OBJECT</code>	Meta-object desteği; her açılan sınıfta zorunlu.
<code>QML_ELEMENT</code>	Sınıfı QML'e kaydeder.
<code>QML_SINGLETON</code>	Tek örnekli tip.
<code>QML_UNCREATABLE</code>	Görünür ama QML'de yaratılamaz (enum açmak için ideal).
<code>Q_PROPERTY</code>	Veriyi özellik olarak açar (READ/WRITE/NOTIFY).
<code>Q_INVOKABLE</code>	Metodu QML'den çağrılabilir yapar.
<code>Q_ENUM</code>	Enum'u QML'e açar.
<code>Q_GADGET</code>	Hafif, kopyalanabilir değer tipi.
<code>signals:</code>	Sinyal bölümü; C++'tan QML'e olay bildirimi.

Özet: QML arayüzü, C++ mantığı. İkisini `Q_PROPERTY` (veri), `Q_INVOKABLE` (metot), **signal** (olay) üçlüsüyle birbirine bağlayın. Tipleri `QML_ELEMENT` ile modül içinde kaydedin, global servisler için singleton kullanın, listeler için `QAbstractListModel` türetin. Akışı tek yönlü tutun: *C++ iter, QML bağlanır*.